GitHub

GitHub Copilot prompt engineering




PROVIDER
Akshay Hosur
SE, Canarys Automations

Our agenda

Prompt engineering introduction

Context in GitHub Copilot information and window size

Best practices improving your prompts

Prompt engineering techniques

Demos prompts in practice

Copilot Customization

GitHub Copilot prompt engineering

Introduction



Definition of a prompt

A prompt is the **collection of code and supporting contextual information** that the GitHub Copilot extension sends to GitHub to generate suggestions.

WHAT IS PROMPT ENGINEERING?

Prompt engineering is the process of designing and refining prompts given to **large language models** to achieve specific, desired outputs.

Key aspects

- **Prompt design**: creating clear and unambiguous prompts
- **Iterative testing**: refine prompts through trial and error
- **Domain-specific knowledge**: tailor prompts to specific topics
- **Bias mitigation**: being aware to ensure fair and balanced responses from the model
- **Constraints and guidelines**: set guardrails within the prompt
- **Use of meta-prompts**: prompts about how to generate prompts
- **Evaluation metrics**: establish quality criteria

Prompt quality for better predictions

Q: “How do I write this Query?”

A: “You can write a Query by writing the necessary code and setting up your server to handle requests”

Vague prompt

- General
- Non-actionable
- Too broad
- Not very helpful

Q: “How do I write a procedure in Snowflake Javascript?”

A: “Create a javascript stored procedure using create procedure and specify language Javascript, use the AS block to write the Javascript logic”

General prompt, little context

- Some context (framework)
- Narrows down response
- Still lacks depth

Q: “How do I create a Snowflake JavaScript stored procedure to retrieve the top 10 customers by total sales? Use the orders table with customer_id and order_amount columns, handle SQL execution properly, and explain each step”

A: “Define the stored procedure using create procedure. Specify language javascript. Execute the sql query using snowflake.createstatement() and execute().

return the results in a structured format.

Specific prompt, detailed context

- Clear, detailed and comprehensive
- Instructions and code samples
- More accurate

Cornerstones of prompting

Help GitHub Copilot help you



Context

Provide examples

Information to help GitHub Copilot understand the task better.



Intent

Give directions

The specific goal or purpose you have in mind when creating a prompt.



Clarity

Easy to understand

Being clear, transparent, and easily understood.



Specificity

Precision to detail

Providing precise and detailed information, leaving little room for ambiguity or interpretation.

GitHub Copilot prompt engineering

Best practices



Prompting best practices



Provide references

Improve relevance of the response by providing an example and context



Write clear instructions

Refine your prompt, provide context, write clearly, and give Copilot ample input for better results



Split up big tasks

Breaking down complex tasks minimizes errors and utilizes previous outcomes for efficiency



Follow good coding practices

Use a consistent code style and patterns, use descriptive names for variables and function



Test changes systematically

Measure performance, watch for prompt changes' side effects, and use test suites for implementation

GitHub Copilot prompt engineering

Techniques



Shot prompting



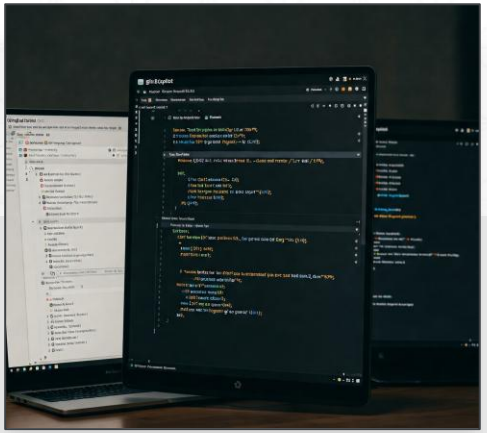
Zero shot prompting

No examples



One shot prompting

One examples



Few shot prompting

Handful of examples

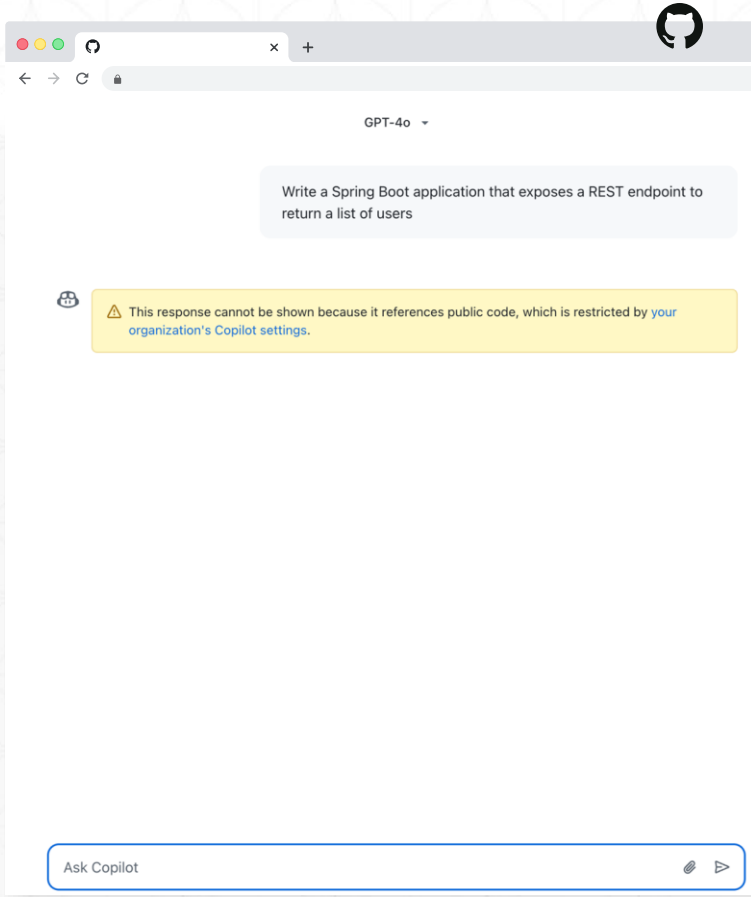
Zero-shot prompting

Definition

Given a task without any examples

Prompt example

“Write a Spring Boot application that exposes a REST endpoint to return a list of users.”



One-shot prompting

Definition

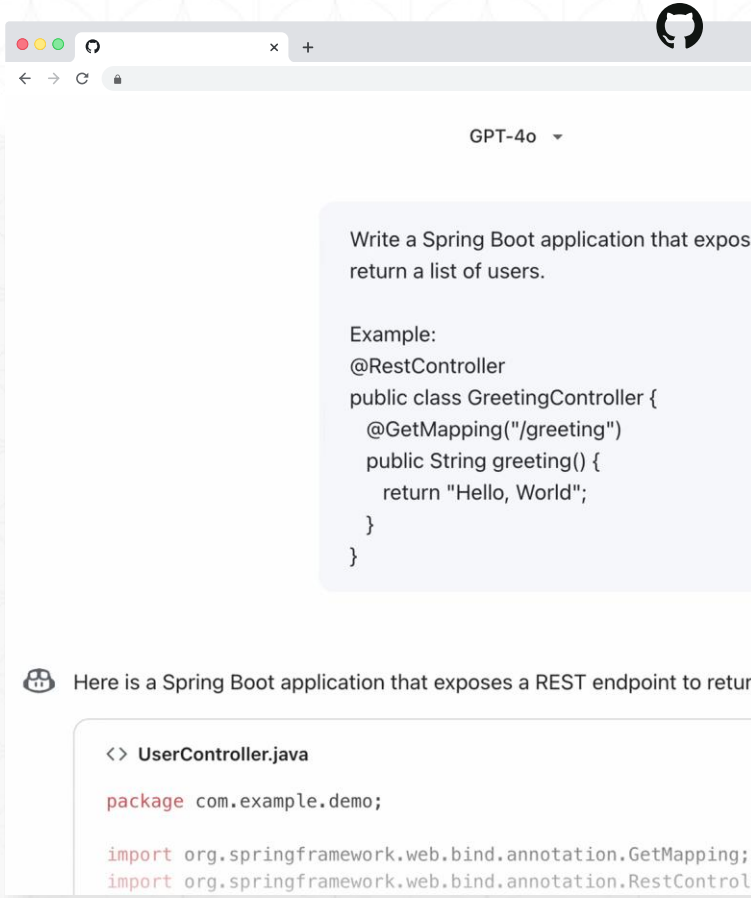
Given a single example along the prompt.

Prompt example

“Write a Spring Boot application that exposes a REST endpoint to return a list of users. Use the following example.”

```
Example:
Write a Spring Boot application with a REST endpoint that returns a greeting message.
@RestController
public class GreetingController {
    @GetMapping("/greeting")
    public String greeting() {
        return "Hello, World!";
    }
}

Now, write a Spring Boot application that exposes a REST endpoint to return a list of user
```



Few-shot prompting

Definition

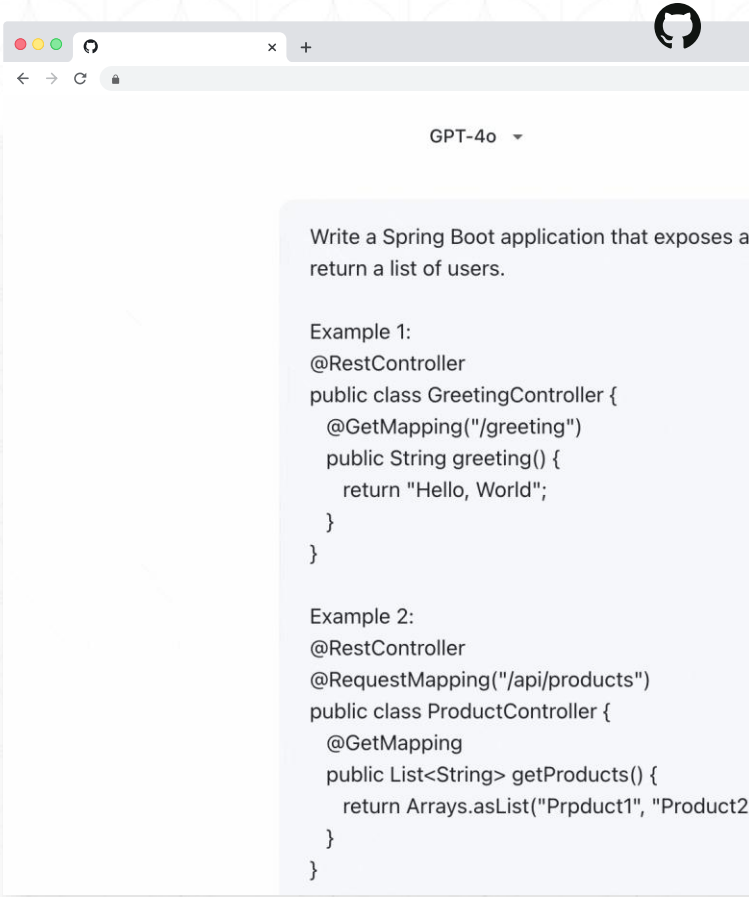
Given a few examples to help set the boundaries.

Prompt example

“Write a Spring Boot application that exposes a REST endpoint to return a list of users. Use the following examples.”

```
Example 1:
Write a Spring Boot application with a REST endpoint that returns a greeting message.
@RestController
public class GreetingController {
    @GetMapping("/greeting")
    public String greeting() {
        return "Hello, World!";
    }
}

Example 2:
Write a Spring Boot application with a REST endpoint that returns a list of products.
@RestController
@RequestMapping("/api/products")
public class ProductController {
    @GetMapping
    public List<String> getProducts() {
        return Arrays.asList("Product1", "Product2", "Product3");
    }
}
```



Chain-of-thought

Provide a step-by-step reasoning process to guide the LLM to a final output

Initial prompt

*“Let’s build a Snowflake pipeline **step by step** that loads daily claims files from S3 into a curated reporting table*

***Step 1:** Create the file format, external stage and raw landing table.”*

Output

```
-- Step 1: File format, stage & landing table
CREATE OR REPLACE FILE FORMAT raw.claims_csv_ff
  TYPE = CSV
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"';

CREATE OR REPLACE STAGE raw.claims_stage
  URL = 's3://claims-landing/daily/'
  STORAGE_INTEGRATION = s3_claims_int
  FILE_FORMAT = raw.claims_csv_ff;

CREATE OR REPLACE TABLE raw.claims (
  claim_id    STRING,
  policy_id   STRING,
  claim_amt   NUMBER(12,2),
  claim_dt    DATE,
  status      STRING
);
```

Chain-of-thought

Provide a step-by-step reasoning process to guide the LLM to a final output

Follow-up prompts

*“**Step 2:** Load the staged files with COPY INTO.”*
*“**Step 3:** Create a stream to capture new rows.”*
*“**Step 4:** MERGE the stream into curated.claims.”*
*“**Step 5:** Schedule the merge with a task.”*

Output

```
-- Step 2: Load the staged files
COPY INTO raw.claims FROM @raw.claims_stage
  ON_ERROR = 'CONTINUE';

-- Step 3: Capture new rows
CREATE OR REPLACE STREAM raw.claims_strm
  ON TABLE raw.claims;

-- Steps 4-5: MERGE, scheduled by a task
CREATE OR REPLACE TASK curated.merge_claims
  WAREHOUSE = etl_wh
  SCHEDULE = 'USING CRON 0 6 * * * UTC'
  WHEN SYSTEM$STREAM_HAS_DATA('raw.claims_strm')
AS
MERGE INTO curated.claims t
USING raw.claims_strm s ON t.claim_id = s.claim_id
WHEN MATCHED THEN UPDATE
  SET t.status = s.status, t.claim_amt = s.claim_amt
WHEN NOT MATCHED THEN INSERT
  (claim_id, policy_id, claim_amt, claim_dt, status)
  VALUES (s.claim_id, s.policy_id, s.claim_amt,
          s.claim_dt, s.status);

ALTER TASK curated.merge_claims RESUME;
```

GitHub Copilot fundamentals

Context engineering



DEVELOPER WORKFLOWS

Context Engineering

Structuring code and habits to maximize GitHub Copilot effectiveness.



Project & Code

Descriptive file paths. Explicit type annotations.

Workspace Habits

Keep relevant tabs open. Position cursor strategically. Chat for complex tasks.

Explicit Hints

Reference exact codebase patterns. Add strategic comments. Add files into copilot context

Managing Complexity

Define scope upfront. Ask for necessary files. Verify changes incrementally.

Context Engineering vs. Prompt Engineering

Feature	Prompt Engineering	Context Engineering
Primary Focus	Refining instructions and formatting queries.	Supplying the right external data at the right time.
Scale	Static, typically designed at the individual prompt level.	Dynamic, designed at the system or application architecture level.
Main Goal	Guiding the model's tone, formatting, and reasoning.	Grounding the model in factual, up-to-date, or private data to prevent hallucinations.

GitHub Copilot fundamentals

Tips and Tricks



DATA ENGINEERING WORKFLOWS

Context Engineering for Snowflake

Structuring SQL, Snowpark code, and data environments to maximize AI effectiveness.



FOUNDATION

Project Structure

Descriptive Paths

Use models/marts/finance/ over sql/queries/. Paths infer domain intent.

Artifacts

Keep SQL models, YAML configs, and Snowpark UDFs together. Group by data domain.

Modularize Logic

Break large scripts into modular CTEs or dbt models to define clear dependencies.

SYNTAX

Code Patterns

Explicit Schemas

Define Snowpark schemas explicitly or use SQL casts like ::VARCHAR for clear typing.

Semantic Naming

Use active_retail_users over temp_tbl. Self-documenting tables are AI-readable.

Use Variables

Use variables or Snowflake session variables instead of hardcoded magic numbers.

EDITOR HABITS

Working with Copilot

Keep DDL Open

Copilot reads open tabs. Open table schemas and YAML files when writing queries.

Position Cursor

Copilot prioritizes code near your cursor. Place it exactly where the JOIN happens.

Use Chat for SQL

Use Chat for complex window functions or translating Pandas to Snowpark Python.

DOCUMENTATION

Context Hints

Add instructions.md

Document Snowflake RBAC rules, warehouse sizing, and naming conventions for the AI.

Comment CTEs

Briefly describe the transformation logic at the top of complex CTEs or Python UDFs.

Reference Models

State "Follow the pattern in stg_customers.sql" to give a concrete standard.

WORKFLOW

Multi-File Changes

Describe Pipeline First

List all staging and mart models involved before requesting schema changes.

Work Incrementally

Modify and test one CTE or Snowpark module at a time. Don't ask for the full DAG at once.

Check Understanding

Ask "What table DDLs do you need to see?" before complex data refactors.

TROUBLESHOOTING

When Copilot Struggles



Missing Schemas

Paste specific Snowflake DDLs or table definitions directly into Chat to ground it.



Stale Suggestions

Copilot may miss recent dbt macro changes. Re-open files or restart the session.



Generic Dialects

Specify "Snowflake SQL" or "Snowpark Python" to avoid generic Postgres or Pandas syntax.

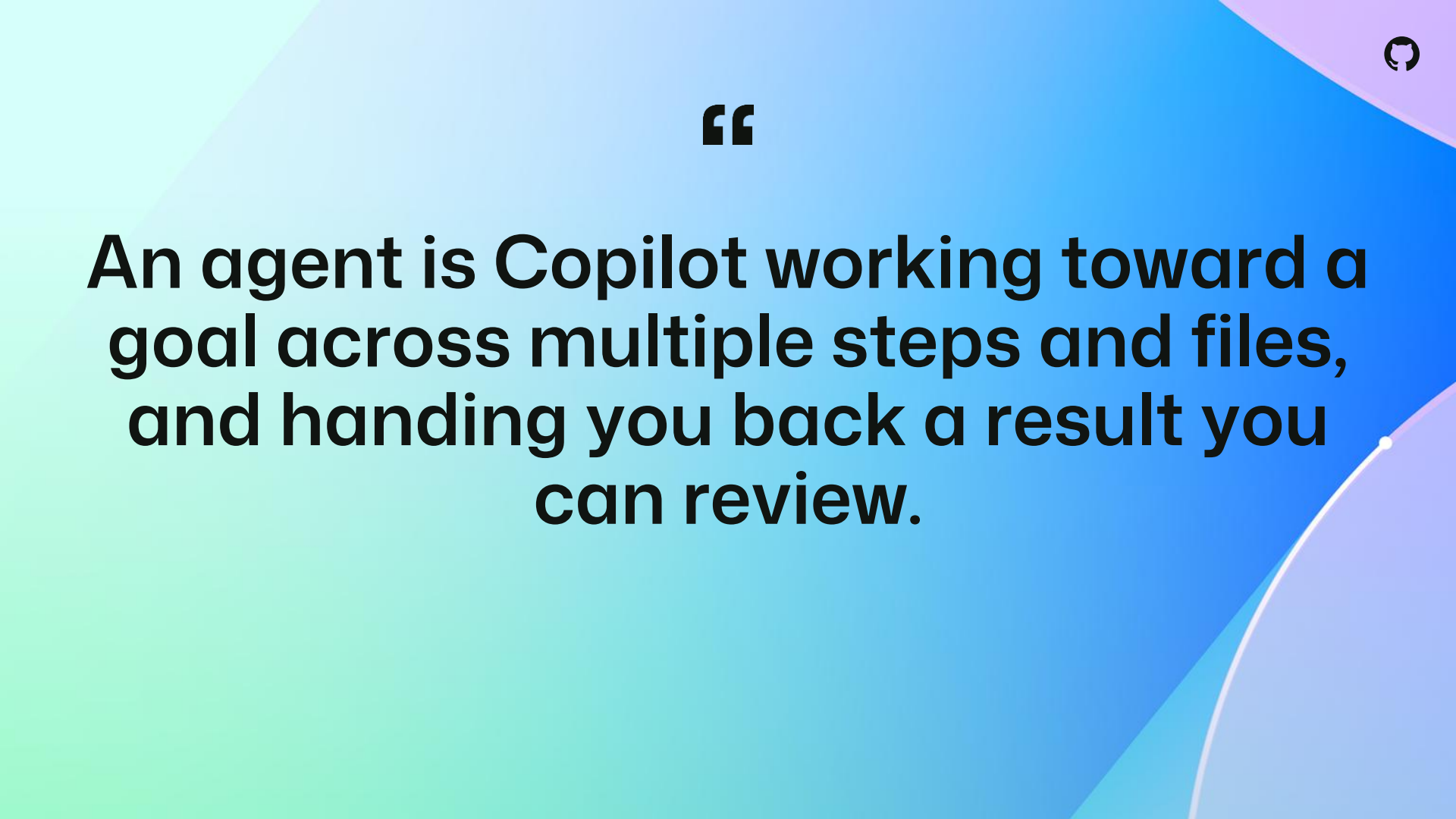
GitHub Copilot fundamentals

Agents

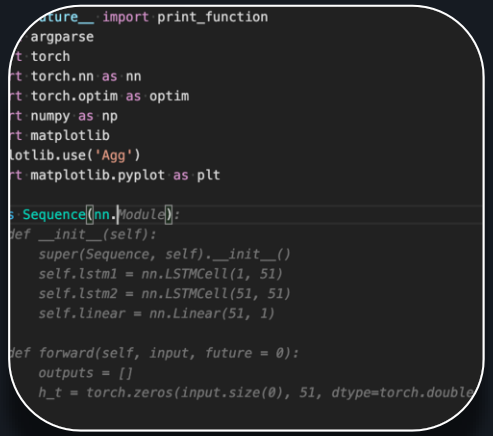


What is an Agent?




“
An agent is Copilot working toward a goal across multiple steps and files, and handing you back a result you can review.

From suggestions to delegations



Autocomplete + Chat

Copilot used to help you write code faster



Agents

Agents help you move work forward without you driving every stroke



Copilot Cloud Agent

Delegate tasks → agent researches, plans, changes code → you review and merge

Works in the background

Own dev environment

Researches + plans

Understands repo before changing code

Edits + validates

Code, tests, docs, merge conflicts

Automates work

Schedule/event-driven tasks, REST APIs, session logs

Custom Agents

Configuration files, with a specialized task

Uses tools and MCP servers

Instructions that guide the conversation style

Guardrails

Agent skills

- Store the steps, examples, and helper scripts for a task in your repo so Copilot can reuse them
- Skills are pulled in when they match your request
- Copilot stays lightweight for simple questions but gets “workflow-aware” for bigger tasks

Agent Instructions

- Provide background context that Copilot reads whenever it works on matching files

They often contain:
- Coding standards or style guides (naming conventions, testing strategy)
- Framework-specific hints
- Repository-specific rules

How it works together

Instructions – Define persistent guidelines and guardrails that guide Copilot's behaviour.

Skills – Package reusable workflows that Copilot can invoke on demand or discover automatically.

Agents – Define specialized personas with tailored instructions, tools, and behaviours for specific tasks.

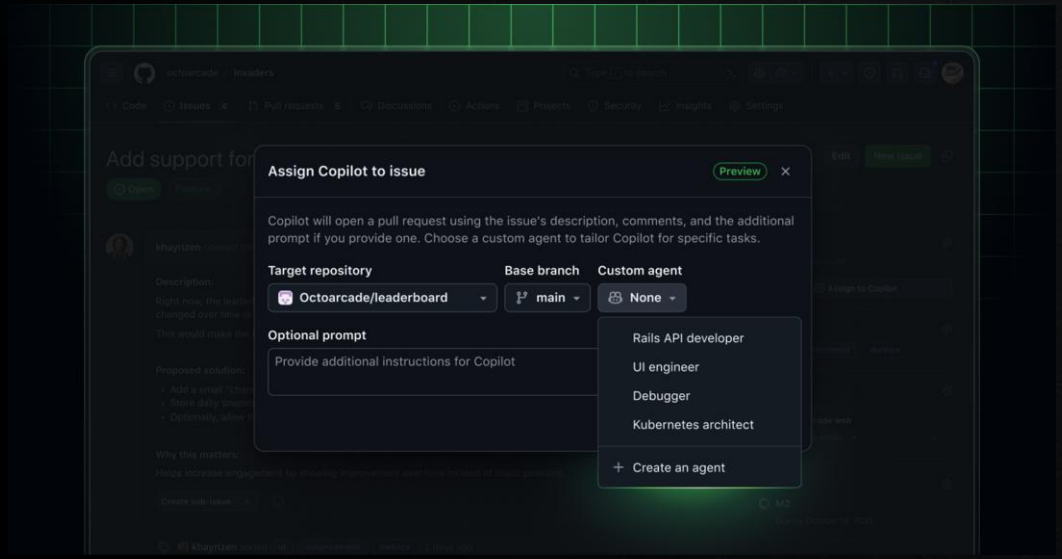
Instructions, skills and agents: what goes where

Three kinds of file in the repo that give Copilot context. They differ in when Copilot reads them and what they hold.

	Custom instructions *The house rules*	Skills *The playbooks*	Custom agents *The specialist teammates*
What it is	Standards and conventions Copilot should always follow in this repo	Step-by-step know-how for one task, plus the templates and scripts it needs	A persona with its own role, tools and model, built for one kind of job
File	`.github/copilot-instructions.md` `.github/instructions/*.instructions.md`	`.github/skills/<name>/SKILL.md` `+ templates, scripts, examples`	`.github/agents/<name>.agent.md`
When it's used	Always on. Added to every request automatically; path-specific files apply when the file matches applyTo	On demand. Copilot loads a skill only when your request matches its description	When you pick it from the agent dropdown in Chat (or another agent hands off to it)
Team example	Snowflake SQL standards: naming, JavaScript stored procedures, the 5 history tracking columns	create-history-load: template procedure for staging → history (SCD2) → processing, with validation queries	snowflake-developer: reads the Jira ticket via MCP, plans, writes DDL and procedures, uses the skills above

Instructions say how we always work · **Skills** say how to do a specific task · **Agents** say who does the work, with which tools

Custom Agents



UI Engineer

Builds UI changes that match your component patterns

Debugger

Finds the root cause and proposes the smallest safe fix.

Kubernetes Architect

Makes deployable cluster-ready changes with sane defaults that match internal templates, tone and level of detail

Agents are everywhere

GitHub.com

Under the Agent HQ page

Your IDE

The newest version of your IDE as well as any other agentic tools needed.

GH Mobile

Can start agents created from mobile!

GH CLI

“gh agent-task create” and more available to start coding-agent sessions from your terminal

Demo

GitHub Copilot prompt engineering

Conclusion



Questions?



GitHub Copilot Session feedback


Thank you